

Investigación REACT: ➤ ¿Qué es Jest y React Testing Library? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Jest es un framework de pruebas en JavaScript desarrollado por Meta, diseñado para asegurar la corrección de cualquier código web. Proporciona el entorno de ejecución, la librería de aserciones (expect) y herramientas avanzadas como *mocking* y pruebas de instantáneas (*snapshots*).

React Testing Library (RTL) es una librería complementaria construida sobre DOM Testing Library. Su propósito es probar componentes de React desde la perspectiva del usuario final (caja negra), fomentando buenas prácticas de accesibilidad y evitando validar detalles de implementación interna (como estados o métodos privados).

Características Principales y Propósito

- **Enfoque centrado en el usuario:** RTL prioriza consultas basadas en accesibilidad y elementos visibles (`getByRole`, `getByText`) en lugar de selectores de clases o IDs.
- **Velocidad y aislamiento:** Jest ejecuta pruebas de forma concurrente optimizando el rendimiento, ideal para integración continua.
- **Simulación realista de eventos:** Utiliza utilidades como `@testing-library/user-event` para replicar interacciones humanas genuinas (clics, escritura, navegación por teclado).
- **Ausencia de dependencias de estado interno:** Las pruebas no dependen de la estructura interna del componente, lo que reduce la fragilidad ante refactorizaciones

Ejemplo:

```
import { useState } from 'react';
```

```
export function Contador() {
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```
    <div>
```

```

    <p>Clicks: {count}</p>
    <button onClick={() => setCount(count + 1)}>Incrementar</button>
  </div>
);
}

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Contador } from './Contador';

test('incrementa el contador al hacer clic en el botón', async () => {
  // 1. Renderizar el componente en un DOM virtual
  render(<Contador />);

  // 2. Buscar elementos usando selectores accesibles
  const boton = screen.getByRole('button', { name: /incrementar/i });
  const textoInicial = screen.getByText(/Clicks: 0/i);

  expect(textoInicial).toBeInTheDocument();

  // 3. Simular la interacción del usuario
  await userEvent.click(boton);

  // 4. Verificar que el estado y la interfaz se actualizaron
  expect(screen.getByText(/Clicks: 1/i)).toBeInTheDocument();
});

```